

PROJECT REPORT

MOVIE RECOMMENDATION SYSTEM

A Content-Based Movie Recommendation System Using TF-IDF and Cosine
Similarity

Submitted by:

Name: _____

Roll No.: _____

Course / Department: _____

College / University: _____

Under the Guidance of:

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled “Movie Recommendation System” has been prepared as an academic project based on the supplied Python/Jupyter Notebook implementation. The work has been carried out for the purpose of studying data preprocessing, text feature extraction, similarity analysis, and recommendation-system concepts.

Guide/Faculty Signature: _____

Name: _____

Department: _____

Date: _____

DECLARATION

I hereby declare that the project report titled “Movie Recommendation System” is prepared for academic purposes and represents my work based on the supplied Python/Jupyter Notebook implementation. The report describes the methods, workflow, algorithms, implementation, results, limitations, and possible future improvements of the system. Wherever external concepts or software libraries are discussed, they are identified appropriately in the references section.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teachers, guide, department, and institution for providing the opportunity and support required to complete this project titled “Movie Recommendation System.” I am thankful for the guidance received during the study of Python, data preprocessing, machine learning concepts, and recommendation systems. I also acknowledge the open-source Python ecosystem and the libraries used in the implementation, particularly NumPy, Pandas, scikit-learn, and Python’s built-in difflib module. Finally, I thank my classmates, friends, and family for their encouragement and support throughout the completion of this project.

TABLE OF CONTENTS

1. ABSTRACT
2. ACKNOWLEDGEMENT
3. DECLARATION
4. CHAPTER 1 — INTRODUCTION
5. 1.1 Background
6. 1.2 Problem Statement
7. 1.3 Objectives
8. 1.4 Scope
9. CHAPTER 2 — LITERATURE AND CONCEPTUAL REVIEW
10. 2.1 Content-Based Filtering
11. 2.2 TF-IDF
12. 2.3 Cosine Similarity
13. 2.4 Title Matching
14. CHAPTER 3 — SYSTEM ANALYSIS
15. 3.1 Functional Requirements
16. 3.2 Non-Functional Requirements
17. 3.3 Hardware Requirements
18. 3.4 Software Requirements
19. CHAPTER 4 — DATASET DESCRIPTION
20. 4.1 Data Preprocessing
21. 4.2 Example Dataset Records
22. CHAPTER 5 — METHODOLOGY
23. 5.1 System Workflow
24. 5.2 Feature Engineering
25. 5.3 Vectorization
26. CHAPTER 6 — SYSTEM DESIGN
27. 6.1 Architecture
28. 6.2 Data Flow
29. CHAPTER 7 — IMPLEMENTATION

30.	7.1 Important Code Components
31.	7.2 Pseudocode
32.	CHAPTER 8 — ALGORITHMS
33.	CHAPTER 9 — RESULTS AND DISCUSSION
34.	9.1 Interpretation of Results
35.	9.2 Sample Output Discussion
36.	CHAPTER 10 — ADVANTAGES AND LIMITATIONS
37.	CHAPTER 11 — FUTURE SCOPE
38.	CHAPTER 12 — CONCLUSION
39.	REFERENCES
40.	APPENDIX A — PROJECT MODULE SUMMARY
41.	APPENDIX B — VIVA QUESTIONS AND ANSWERS

ABSTRACT

The Movie Recommendation System is a content-based recommendation project implemented in Python. The system uses information associated with movies to identify movies that are similar to a movie selected by the user. The notebook loads a movie dataset containing 4,803 records and 24 columns, selects five recommendation-oriented attributes—genres, keywords, tagline, cast, and director—and replaces missing values in these fields with empty strings. These fields are then combined into a single textual representation for every movie. TF-IDF Vectorization converts the combined textual information into numerical feature vectors. Cosine similarity is subsequently used to calculate the similarity between every pair of movie vectors. When a user enters a favourite movie name, the system uses the available movie-title list and title matching logic to identify the closest corresponding movie and uses the similarity scores to determine related movies. The project demonstrates a practical application of natural language processing and machine learning concepts for personalized information retrieval. It is lightweight, understandable, and suitable as an academic demonstration of a content-based recommendation engine.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teachers, guide, department, and institution for providing the opportunity and support required to complete this project titled “Movie Recommendation System.” I am thankful for the guidance received during the study of Python, data preprocessing, machine learning concepts, and recommendation systems. I also acknowledge the open-source Python ecosystem and the libraries used in the implementation, particularly NumPy, Pandas, scikit-learn, and Python’s built-in difflib module. Finally, I thank my classmates, friends, and family for their encouragement and support throughout the completion of this project.

DECLARATION

I hereby declare that the project report titled “Movie Recommendation System” is prepared for academic purposes and represents my work based on the supplied Python/Jupyter Notebook implementation. The report describes the methods, workflow, algorithms, implementation, results, limitations, and possible future improvements of the system. Wherever external concepts or software libraries are discussed, they are identified appropriately in the references section.

CHAPTER 1 — INTRODUCTION

Recommendation systems are software systems that help users discover relevant items from a larger collection. In the entertainment domain, a movie catalogue may contain thousands of titles, making it difficult for a user to decide what to watch. A recommendation system can reduce this search effort by presenting movies related to a title the user already likes.

This project develops a content-based Movie Recommendation System. Instead of requiring a history of ratings from many users, the implemented approach compares descriptive information about movies. The supplied notebook works with a movie dataset and selects genres, keywords, tagline, cast, and director as the features used to represent each movie. These textual attributes are combined and transformed into numerical vectors using TF-IDF. Similarity between movie vectors is then calculated using cosine similarity.

The project is useful for understanding the complete basic workflow of a machine-learning-oriented data project: importing dependencies, loading a CSV dataset, inspecting the data, selecting relevant attributes, handling missing values, constructing features, vectorizing text, computing similarity, accepting user input, and producing recommendations.

1.1 Background

The growth of digital entertainment platforms has created very large catalogues of movies. A simple search interface requires the user to know what title to search for, whereas a recommendation system can start from an item that the user already knows and identify related items automatically.

Content-based filtering is particularly appropriate for an introductory recommendation project because it can work directly with item attributes. If two movies contain similar genres, keywords, cast information, taglines, or directors, their feature representations may be close in vector space. A similarity score can therefore be used as a measure of relatedness.

1.2 Problem Statement

Users often face difficulty selecting a movie from a large catalogue. The problem addressed by this project is to design a system that accepts a movie title from a user and recommends other movies that are similar according to descriptive movie information available in the dataset.

1.3 Objectives

The main objectives are:

- To load and inspect a structured movie dataset.
- To select meaningful movie attributes for content-based recommendation.
- To handle missing values in the selected attributes.
- To combine the selected attributes into a unified text representation.
- To transform text into numerical feature vectors using TF-IDF.
- To compute movie-to-movie similarity using cosine similarity.
- To accept a movie title from the user and identify the closest matching title.
- To use similarity scores to generate movie recommendations.
- To demonstrate an understandable machine-learning workflow using Python.

1.4 Scope

The scope of the project is a content-based recommendation engine operating on the supplied movie dataset. It focuses on similarity derived from movie metadata rather than collaborative user-rating behaviour. The system is intended as an academic prototype and can be extended into a web or desktop application in future work.

CHAPTER 2 — LITERATURE AND CONCEPTUAL REVIEW

Recommendation systems are commonly categorized into content-based filtering, collaborative filtering, and hybrid approaches. Content-based systems compare item attributes. Collaborative systems learn patterns from

interactions between users and items. Hybrid systems combine multiple approaches.

For this project, content-based filtering is selected because the supplied dataset contains descriptive metadata that can be converted into text features. TF-IDF is a standard information-retrieval technique for assigning importance to terms in documents. In this context, each movie can be treated as a document composed of its selected metadata. Cosine similarity is then used to compare the resulting vectors.

The project therefore combines ideas from information retrieval, natural language processing, and machine learning into a single recommendation workflow.

2.1 Content-Based Filtering

In content-based filtering, an item is represented using its characteristics. A user's selected item is used as a query, and other items with similar characteristics are ranked highly. In the present project, the item characteristics are genres, keywords, tagline, cast, and director.

2.2 TF-IDF

TF-IDF stands for Term Frequency–Inverse Document Frequency. It represents the importance of a word in a document relative to a collection of documents. Terms that occur frequently in one document but are relatively less common across the collection can receive greater importance. The notebook creates a `TfidfVectorizer` object and applies it to the combined movie features, producing numerical feature vectors.

2.3 Cosine Similarity

Cosine similarity measures the angle between two vectors. For vectors A and B , the commonly used formulation is: $\text{cosine}(A,B) = (A \cdot B) / (||A|| ||B||)$. A larger value indicates greater directional similarity. The notebook applies `cosine_similarity` to the complete feature-vector matrix, producing a $4,803 \times 4,803$ similarity matrix.

2.4 Title Matching

The notebook obtains the list of all movie titles and accepts a movie name from the user. Python's `difflib` functionality is imported as part of the project dependencies and is used for title matching so that an entered title can be associated with a title in the dataset even when the input is not an exact textual match.

CHAPTER 3 — SYSTEM ANALYSIS

System analysis identifies the inputs, processing steps, outputs, and constraints of the proposed solution. The system takes a movie title as its principal user input. The dataset provides metadata required to represent each movie. Processing consists of cleaning selected attributes, combining them, vectorizing the text, calculating similarities, matching the requested title, and ranking similar movies.

The approach is deterministic for a fixed dataset and vectorization configuration. It does not require model training with labelled target values. Instead, it constructs a feature representation and uses similarity calculations.

3.1 Functional Requirements

1. The system shall load the movie dataset from a CSV file.
2. The system shall inspect the movie records.
3. The system shall select the five relevant content features.
4. The system shall replace null values in those selected features.
5. The system shall combine the selected features.
6. The system shall create TF-IDF feature vectors.
7. The system shall calculate cosine similarity.
8. The system shall accept a movie title from the user.
9. The system shall identify a matching movie title.
10. The system shall provide similar movie recommendations.

3.2 Non-Functional Requirements

The prototype should be understandable, reproducible, and reasonably efficient for the supplied dataset. It should provide a simple user interaction through text input. It should also be maintainable so that additional features or a different dataset can be incorporated later.

3.3 Hardware Requirements

A standard personal computer is sufficient for the academic prototype. Recommended resources include a modern multi-core processor, at least 4 GB RAM, sufficient storage for Python and the dataset, and a keyboard/display for interacting with the notebook. Higher memory may be beneficial when working with larger datasets or larger similarity matrices.

3.4 Software Requirements

- Python 3
- Jupyter Notebook or Google Colab
- NumPy
- Pandas
- scikit-learn
- difflib
- CSV movie dataset

The supplied notebook identifies a Python 3 kernel and imports NumPy, Pandas, difflib, TfidfVectorizer, and cosine_similarity.

CHAPTER 4 — DATASET DESCRIPTION

The supplied notebook loads movie data from “/content/movies.csv” into a Pandas DataFrame. The dataset contains 4,803 rows and 24 columns. The displayed columns include index, budget, genres, homepage, id, keywords, original_language, original_title, overview, popularity, production_companies, production_countries, release_date, revenue, runtime, spoken_languages, status, tagline, title, vote_average, vote_count, cast, crew, and

director.

Although the dataset contains many fields, the recommendation calculation uses five selected attributes: genres, keywords, tagline, cast, and director. This selection focuses the similarity computation on descriptive and creator-related information rather than financial or popularity fields.

4.1 Data Preprocessing

Missing values in each selected feature are replaced with an empty string. This prevents null values from interfering with the concatenation step. The five selected columns are then joined with spaces to produce one combined feature string for each movie.

4.2 Example Dataset Records

The notebook displays the first five records. Examples include Avatar, Pirates of the Caribbean: At World's End, Spectre, The Dark Knight Rises, and John Carter. The records demonstrate that the selected metadata fields contain genre information, keywords, taglines, cast members, and director names.

CHAPTER 5 — METHODOLOGY

The methodology follows a sequence of data preparation, feature engineering, vectorization, similarity computation, title matching, and recommendation.

Step 1: Import dependencies.

Step 2: Load movies.csv into a Pandas DataFrame.

Step 3: Inspect the dataset and its dimensions.

Step 4: Select genres, keywords, tagline, cast, and director.

Step 5: Replace null values with empty strings.

Step 6: Concatenate the five attributes into combined_features.

Step 7: Create TfidfVectorizer.

Step 8: Fit and transform combined_features into feature_vectors.

Step 9: Calculate cosine similarity for all feature vectors.

Step 10: Ask the user for a favourite movie name.

Step 11: Compare the input with the dataset's movie-title list.

Step 12: Use the identified movie and similarity scores to obtain recommendations.

5.1 System Workflow

User enters movie title → Title matching → Identify movie index → Retrieve similarity scores → Rank similar movies → Display recommendations.

The preprocessing and model-building path is performed before the user query. The title-query path then uses the precomputed similarity information to identify movies related to the selected title.

5.2 Feature Engineering

Feature engineering is central to this project. The system does not directly use every dataset column. Instead, it constructs a single textual feature for each movie by joining genres, keywords, tagline, cast, and director. This makes it possible to apply a text vectorization method to heterogeneous metadata.

5.3 Vectorization

The `TfidfVectorizer` converts combined movie descriptions into a sparse numerical matrix. Each row represents a movie and each column corresponds to a term in the generated vocabulary. The numerical values indicate the TF-IDF weight of a term for a movie.

CHAPTER 6 — SYSTEM DESIGN

The system can be represented as five logical modules: Data Input, Preprocessing, Feature Extraction, Similarity Engine, and Recommendation Interface.

Data Input reads the CSV file. Preprocessing selects and cleans the required fields. Feature Extraction concatenates metadata and generates TF-IDF vectors. The Similarity Engine computes cosine similarity. The Recommendation Interface receives a movie title and returns similar titles.

This modular view makes the project easy to understand and extend.

6.1 Architecture

Dataset → DataFrame → Selected Features → Missing-Value Handling → Combined Features → TF-IDF Matrix → Cosine Similarity Matrix → User Movie Input → Title Matching → Similarity Ranking → Recommendations.

6.2 Data Flow

The data flows from a static CSV source into Pandas. Selected textual attributes are normalized by filling missing values. These attributes are combined into a single string per movie. TF-IDF transforms the strings into numerical vectors. The similarity engine compares these vectors. Finally, the user query selects one movie vector and the system uses its similarity relationships to identify recommendations.

CHAPTER 7 — IMPLEMENTATION

The implementation is written in Python inside a Jupyter/Colab notebook. The first code section imports NumPy, Pandas, `difflib`, `TfidfVectorizer`, and `cosine_similarity`. The movie dataset is loaded with `pandas.read_csv`. The notebook then examines the first records and reports the dataset shape.

The selected feature list is explicitly defined. A loop fills missing values with empty strings. The selected features are concatenated to form `combined_features`. `TfidfVectorizer` is initialized and `fit_transform` is applied. `cosine_similarity` is then applied to the feature vectors.

For the user interaction, the notebook calls `input()` with the prompt “Enter your favourite movie name”. The displayed test input is “iron man”. The notebook also creates `list_of_all_titles` from the title column, providing

the basis for matching the user's entry.

7.1 Important Code Components

The core implementation components are:

- `pandas.read_csv` — loads the movie dataset.
- `DataFrame.head()` — previews records.
- `DataFrame.shape` — reports dataset dimensions.
- `fillna("")` — handles missing selected feature values.
- `TfidfVectorizer()` — creates the text vectorizer.
- `fit_transform()` — learns the vocabulary and transforms combined features.
- `cosine_similarity()` — computes pairwise similarity.
- `input()` — receives the user's movie title.
- `tolist()` — creates the list of movie titles for matching.

7.2 Pseudocode

BEGIN

Load movie dataset

Select genres, keywords, tagline, cast, director

FOR each selected feature

Replace missing values with empty string

END FOR

Combine selected features

Create TF-IDF vectorizer

Transform combined features into vectors

Compute cosine similarity matrix

Read user's movie name

Match input against available titles

Identify selected movie

Retrieve similarity scores

Rank movies by similarity

Display recommended movies

END

CHAPTER 8 — ALGORITHMS

Algorithm 1: Data Preparation

Input: `movies.csv`

Output: cleaned selected features

1. Read CSV into `DataFrame`.
2. Define `selected_features`.
3. For each selected feature, replace null values with an empty string.
4. Concatenate the five features.
5. Return combined feature strings.

Algorithm 2: Content Similarity

Input: combined feature strings

Output: similarity matrix

1. Initialize TfidfVectorizer.
2. Fit and transform the combined strings.
3. Calculate cosine similarity among all movie vectors.
4. Store the resulting matrix.

Algorithm 3: Recommendation Query

Input: user movie title

Output: similar movies

1. Create the title list.
2. Match the user's title to a dataset title.
3. Obtain the corresponding movie index.
4. Read similarity scores for that movie.
5. Sort the scores in descending order.
6. Select the most similar titles.
7. Display recommendations.

CHAPTER 9 — RESULTS AND DISCUSSION

The notebook successfully constructs the recommendation data pipeline. The dataset shape output is (4803, 24), confirming 4,803 records and 24 columns. The five selected features are printed explicitly. The combined feature series contains 4,803 movie-level text representations.

After TF-IDF transformation, the notebook prints sparse feature-vector values, demonstrating that textual metadata has been converted to numerical weights. Cosine similarity is then computed and the resulting matrix has shape (4803, 4803). Diagonal values are 1.0 because each movie is maximally similar to its own vector.

The notebook demonstrates a user query using the movie name “iron man”. The title list is created from the dataset's title column, allowing the recommendation stage to connect the user's input with a movie record.

9.1 Interpretation of Results

A high cosine similarity indicates that two movies have more similar weighted terms across the selected metadata. For example, movies sharing genre terms, keywords, cast names, or director names may receive higher similarity than movies with unrelated metadata. The recommendation is therefore driven by content similarity rather than popularity or user ratings.

9.2 Sample Output Discussion

The notebook displays the similarity matrix and its dimensions. The first several matrix entries include values such as 1.0 on the diagonal and smaller values for different movies, illustrating the range of similarity relationships. The exact recommendation list should be taken from the executed notebook output when presenting a final demonstration screenshot, rather than being inferred from the source code alone.

CHAPTER 10 — ADVANTAGES AND LIMITATIONS

Advantages:

- Simple and understandable architecture.
- Does not require a user-rating history.
- Uses meaningful movie metadata.
- TF-IDF provides a practical text representation.
- Cosine similarity is straightforward to interpret.
- Can recommend items even for a new user who selects a movie.
- Easy to prototype in Python.

Limitations:

- Recommendations depend strongly on the available metadata.
- The system does not model individual user preferences beyond the selected movie.
- It does not implement collaborative filtering.
- Similarity is based on textual metadata and may miss deeper semantic relationships.
- A large pairwise similarity matrix can require significant memory as the catalogue grows.
- The supplied implementation is a notebook-based prototype rather than a deployed application.
- The project does not document an offline quantitative accuracy evaluation in the supplied notebook.

CHAPTER 11 — FUTURE SCOPE

The project can be extended in several directions. A web interface could be created so users can search for movies and receive recommendations without using a notebook. Movie posters, ratings, release years, and descriptions could be displayed alongside recommendation results.

A hybrid recommendation system could combine content-based similarity with collaborative filtering. More advanced natural language representations, embeddings, or transformer-based models could improve semantic understanding. User accounts and feedback could be added to personalize results over time. The recommendation engine could also use approximate nearest-neighbor search or other scalable retrieval methods instead of storing a complete pairwise similarity matrix for very large datasets.

Evaluation can also be improved by creating a test protocol with relevance judgments and metrics such as Precision@K, Recall@K, or NDCG, provided suitable evaluation data is available.

CHAPTER 12 — CONCLUSION

The Movie Recommendation System demonstrates how machine-learning and natural-language-processing techniques can be combined to create a useful content-based recommendation engine. The project begins with a structured movie dataset and selects five descriptive attributes: genres, keywords, tagline, cast, and director. Missing values are handled, the attributes are combined, and TF-IDF transforms the resulting text into numerical feature vectors. Cosine similarity then measures the relatedness of movies.

The supplied notebook confirms a dataset size of 4,803 records and produces a $4,803 \times 4,803$ similarity matrix. A user can enter a movie title, such as the demonstrated “iron man” query, and the title-matching stage connects the input to the movie catalogue. The resulting similarity relationships are used to generate recommendations.

As an academic project, the system provides a clear demonstration of data preprocessing, feature engineering, text vectorization, similarity computation, and user-oriented recommendation. Its architecture is simple enough to explain in a viva while also providing a strong foundation for future improvements such as web deployment, hybrid filtering, richer semantic models, and formal recommendation evaluation.

REFERENCES

1. Python documentation — Python programming language and standard library.
2. NumPy documentation — numerical computing with Python.
3. Pandas documentation — data analysis and DataFrame operations.
4. scikit-learn documentation — TfidfVectorizer and cosine_similarity.
5. Supplied project notebook: Movie_Recommendation_System.ipynb.
6. General information-retrieval literature concerning TF-IDF and vector-space similarity.

APPENDIX A — PROJECT MODULE SUMMARY

Module 1: Data Loading — reads movies.csv.
Module 2: Data Inspection — previews records and checks dimensions.
Module 3: Feature Selection — selects genres, keywords, tagline, cast, director.
Module 4: Preprocessing — fills missing selected fields.
Module 5: Feature Combination — creates combined_features.
Module 6: TF-IDF — creates feature_vectors.
Module 7: Similarity — creates the cosine similarity matrix.
Module 8: User Input — receives a favourite movie name.
Module 9: Title Matching — works with the dataset title list.
Module 10: Recommendation — ranks related movies by similarity.

APPENDIX B — VIVA QUESTIONS AND ANSWERS

- Q1. What type of recommendation system is this?
A. It is primarily a content-based movie recommendation system.
- Q2. Which dataset is used?
A. The notebook loads movies.csv.
- Q3. How many records are present?
A. The notebook reports 4,803 rows and 24 columns.
- Q4. Which features are used for recommendation?
A. Genres, keywords, tagline, cast, and director.
- Q5. Why are null values replaced?
A. Empty strings allow the selected textual features to be safely combined without null values disrupting the feature construction.
- Q6. What is TF-IDF?
A. It is a text-weighting technique that represents the importance of terms within documents relative to the

collection.

Q7. Why is cosine similarity used?

A. It measures the similarity in direction between feature vectors and is commonly used for comparing text representations.

Q8. What is the size of the similarity matrix?

A. $4,803 \times 4,803$.

Q9. Does the system use user ratings?

A. Not in the supplied implementation; it relies on movie metadata.

Q10. What is one major limitation?

A. Recommendations depend on the quality and coverage of the selected metadata.

Q11. What is title matching used for?

A. It helps associate the user's entered movie name with a title in the dataset.

Q12. How can the project be improved?

A. It can be extended with a web UI, hybrid filtering, semantic embeddings, user feedback, and formal evaluation.